

From regression to classification

Until now you predicted a **number** (apartment rent). Now the target is a **category**:

Is this cheese batch bad?

Running example (the whole chapter): a factory makes 1000 cheese wheels a day; about **3% are bad** (contaminated / spoiled). Missing a bad batch is expensive.

The same shape of problem is everywhere:

- ▶ spam vs not-spam (email text)
- ▶ default vs repay (credit)
- ▶ churn vs stay (customers)
- ▶ sick vs healthy (screening)

good batch (0)

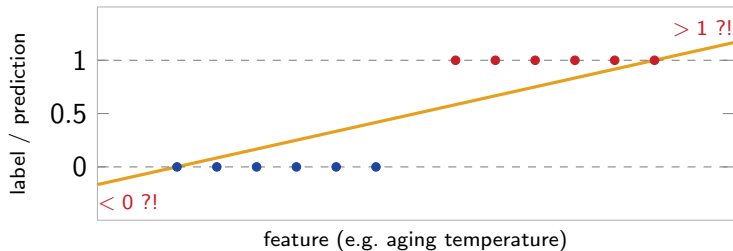
bad batch (1)

target $y \in \{0, 1\}$, not a number

The label is the rare “bad” class (positive, $\sim 3\%$).

Predict-first: just fit a line to the 0/1 labels?

We know linear regression. Why not fit a straight line to labels that are 0 or 1, and read off the value?



It breaks: predictions go **below 0 and above 1** (not probabilities), one far point **tilts the whole line**, and the implied threshold drifts. *We need a model whose output is squeezed into $[0, 1]$ – that is logistic regression.*

Outline

Classification basics

From odds to the sigmoid

Logistic regression as a model

The loss: how we fit it

In practice

Multiclass

Binary and multiclass

Classification = assign each observation to one of a **finite set of unordered classes**.
The target is categorical, not numeric.

Binary (2 classes)

- ▶ cheese: good / **bad**
- ▶ email: ham / spam
- ▶ loan: repay / default

Multiclass (> 2)

- ▶ digit: 0–9
- ▶ iris: setosa / versicolor / virginica
- ▶ sentiment: neg / neutral / pos

This deck: binary logistic regression first, then **multiclass** (one-vs-rest, softmax) later on.

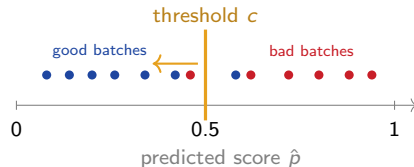
Probabilistic classifiers and the threshold

A good classifier outputs a **score**

$\hat{p}(\mathbf{x}) \in [0, 1]$, not just a label. Threshold it into a decision:

$$\hat{y} = \begin{cases} 1 \text{ (bad)} & \hat{p} \geq c \\ 0 \text{ (good)} & \text{else} \end{cases}$$

Default $c = 0.5$, but c is a **choice**. Missing a bad batch hurts more than a re-check, so for cheese you may slide c *left* (catch more bad). How to pick it: the classification-metrics lecture.



Right of $c \Rightarrow$ predict bad. Sliding c left catches more bad (with more false alarms).

Score vs probability. The number is a *score* that behaves like a probability; whether it is a *trustworthy* probability is a separate question (*calibration*). For now we use it to rank and threshold.

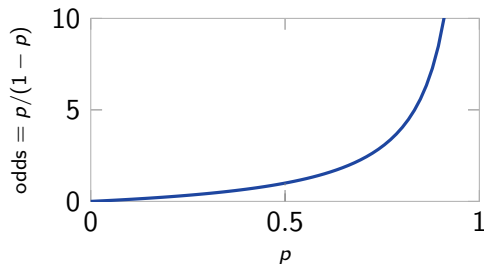
Building the model: odds $\rightarrow$ log-odds $\rightarrow$ sigmoid

A line outputs *any* real number, but a probability must live in $[0, 1]$. Three steps — odds, log-odds, then the sigmoid — bridge the two.

Odds

$$\text{Odds} = \frac{p}{1-p} \quad (p = \text{probability of the positive class}).$$

prob. p	odds	read as
0.5	1	1 : 1 (even)
0.75	3	3 : 1
0.9	9	9 : 1
0.1	1/9	1 : 9



This is betting odds. Bookmakers think in odds, not probabilities – “3 to 1” is the same $p/(1-p)$ (they quote odds *against*, $(1-p)/p$, shaded so the house wins). Probabilities live in $[0, 1]$; odds in $(0, \infty)$ – they *explode* as $p \rightarrow 1$. Not yet a free real number.

Log-odds (the logit)

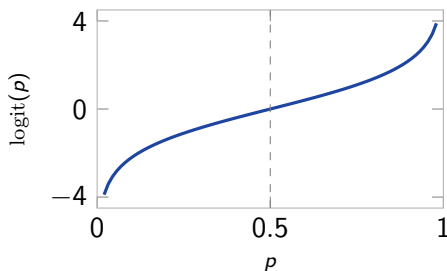
Take the log of the odds:

$$\text{logit}(p) = \log \frac{p}{1-p} \in (-\infty, +\infty).$$

Now the value is a **free real number** – exactly what a linear model outputs.

Why take the log? A linear model *adds* effects up. Logs turn *multiplying* the odds into *adding* a fixed amount, so each coefficient becomes one constant, readable number (an odds ratio) – more later.

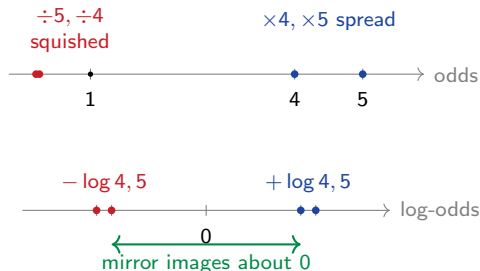
Symmetry: $\text{logit}(p)$ and $\text{logit}(1-p)$ are negatives of each other – $p = 0.5$ sits at 0.



Why the log-odds? A fair, unbounded scale

A linear model $\theta^\top \mathbf{x}$ can output *any* real number, so we want a target on the same scale:

- ▶ $p \in [0, 1]$ — **capped**; a line shoots past it (slide 2).
- ▶ odds $\frac{p}{1-p} \in (0, \infty)$ — removes the *top* cap, but floored at 0 and **lopsided**.
- ▶ log-odds $\in (-\infty, \infty)$ — **free both ways**: the right target.



Fair to both directions. “4× more likely” and “4× less likely” should be equal-and-opposite. On the odds scale they are *not* (4 vs 0.25); on the log scale they are exactly $\pm \log 4$.

Model the log-odds with a line

The whole idea of logistic regression in one line: assume the **log-odds is linear** in the features.

$$\log \frac{p(\mathbf{x})}{1 - p(\mathbf{x})} = z = \theta_0 + \theta_1 x_1 + \cdots + \theta_p x_p = \boldsymbol{\theta}^\top \mathbf{x}.$$

We reuse everything from linear regression (a linear score $\boldsymbol{\theta}^\top \mathbf{x}$), but apply it to the log-odds instead of to y directly.

To turn this score back into a probability $p(\mathbf{x}) \in (0, 1)$, we **invert** it – next.

Invert: derive the sigmoid

Start from the log-odds, **exponentiate** both sides, then solve for p :

$$\begin{aligned}\log \frac{p}{1-p} = z &\Rightarrow \frac{p}{1-p} = e^z \Rightarrow p = (1-p) e^z \\ &\Rightarrow p(1 + e^z) = e^z \Rightarrow p = \frac{e^z}{1+e^z} = \frac{1}{1+e^{-z}} =: \sigma(z).\end{aligned}$$

The **sigmoid** σ squashes any real score $z = \boldsymbol{\theta}^\top \mathbf{x}$ into a probability in $(0, 1)$:

$$z \rightarrow +\infty \Rightarrow p \rightarrow 1; \quad z = 0 \Rightarrow p = 0.5; \quad z \rightarrow -\infty \Rightarrow p \rightarrow 0.$$

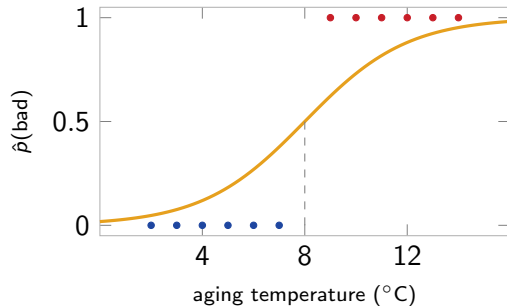
Logistic regression's hypothesis: $\hat{p}(\mathbf{x}) = \sigma(\boldsymbol{\theta}^\top \mathbf{x})$ — a linear score, squashed into a probability.

The sigmoid (logistic) curve

The score $z = \theta^\top \mathbf{x}$ runs along the x-axis; the sigmoid bends it into a probability.

- **Steepest at** $z = 0$ ($p = 0.5$) — the decision point.
- **Flattens** toward 0 and 1 — the confident regions.

Same curve, “logistic growth”: cumulative COVID cases in a wave trace a sigmoid — slow start, explosive middle, then a plateau as susceptibles run out.



Illustrative (balanced for clarity; real data is 3% bad).

What model did we just build?

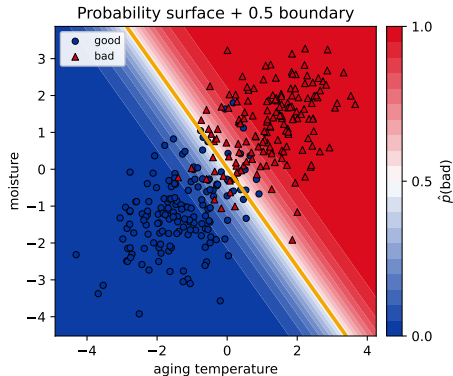
We have $\hat{p} = \sigma(\boldsymbol{\theta}^\top \mathbf{x})$. Now: what is it as a classifier — its decision boundary, and what each weight does to the curve.

Logistic regression is a linear classifier

Prediction: $\hat{p}(\mathbf{x}) = \sigma(\boldsymbol{\theta}^\top \mathbf{x})$. Decide “bad” when $\hat{p} \geq 0.5$, i.e. when $\boldsymbol{\theta}^\top \mathbf{x} \geq 0$.

So the **decision boundary** $\boldsymbol{\theta}^\top \mathbf{x} = 0$ is a straight line (a hyperplane). The sigmoid only bends the *probability*, not the boundary.

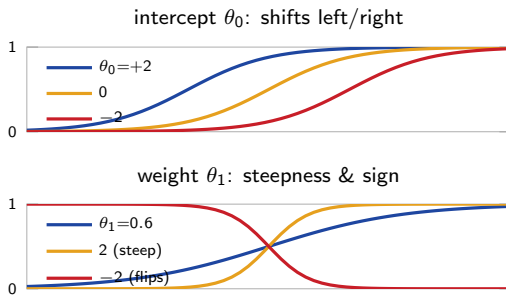
Why is it called “regression”? It *regresses* a continuous quantity – the log-odds – on the features. The *fit* is regression-like; the *task* is classification.



Real fit (illustrative, balanced). Straight boundary ($\boldsymbol{\theta}^\top \mathbf{x} = 0$); “bad” upper-right.

What the parameters do

- **Intercept** θ_0 shifts the S-curve left/right (the baseline level).
- **Weight** θ_1 : *magnitude* = steepness; *sign* = direction (a negative weight flips the curve).
- Boundary:
 $\sigma(\theta^\top \mathbf{x}) = 0.5 \iff \theta^\top \mathbf{x} = 0.$



0.5 is the default threshold, not a law. It is just where $\theta^\top \mathbf{x} = 0$. With asymmetric costs (missing bad cheese!) you will move it – the metrics lecture shows how. The *model* gives the score; *you* pick the cutoff.

Worked numbers (one batch, by hand)

Fitted $\theta_0 = -4$, $\theta_1 = 0.5$ (one feature, aging temp x); take a batch at $x = 10^\circ\text{C}$:

$$z = -4 + 0.5 \cdot 10 = 1$$

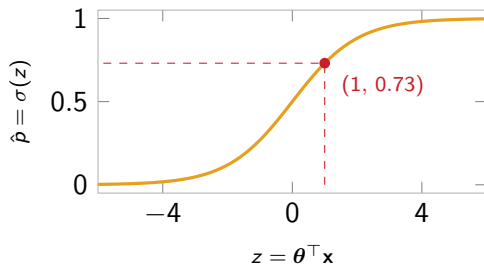
$$\hat{p} = \sigma(1) = \frac{1}{1+e^{-1}} = \mathbf{0.73}$$

$$\text{odds} = \frac{0.73}{0.27} = e^1 = \mathbf{2.7}$$

Slope as an **odds ratio**: $e^{\theta_1} = e^{0.5} \approx \mathbf{1.65}$

– each $+1^\circ\text{C}$ multiplies the odds of “bad” by 1.65 (+65%).

One z , one $\hat{p}$, one odds, one odds ratio – the whole mechanics in four numbers. (*Raw, unscaled features, so θ_1 reads “per $^\circ\text{C}$ ”.*)



How do we fit it?

We have the hypothesis $\hat{p} = \sigma(\boldsymbol{\theta}^\top \mathbf{x})$ but no weights yet. What should we minimize — and why not the squared loss we already know?

Predict-first: why not squared loss?

We have a model $\hat{p} = \sigma(\theta^\top \mathbf{x})$. Could we fit it by minimizing $\sum_i (y^{(i)} - \hat{p}^{(i)})^2$, like linear regression?

Two problems:

- **Non-convex.** Squared loss $\circ$ sigmoid is not convex in θ — gradient descent can get stuck in local minima.
- **Weak penalty.** It is *bounded*: even a confidently wrong $\hat{p}$ costs at most ≈ 1 . Confident mistakes should hurt far more.

True $y = 1$: what each loss charges		
model $\hat{p}$	squared $(1 - \hat{p})^2$	log $-\log \hat{p}$
0.5	0.25	0.69
0.1	0.81	2.30
0.01	0.98	4.61
0.001	≈ 1.00	6.91

Squared **saturates near 1**; log keeps growing $\rightarrow \infty$.

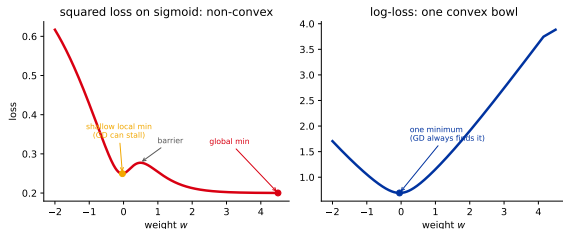
We need a loss built for probabilities — next.

Why not squared loss: the loss surface

Fit a single weight w and watch each loss as w varies:

- **Squared loss** on the sigmoid is **non-convex** — a bump (barrier) can trap gradient descent in a shallow *local* min, away from the real one.
- **Log-loss** is a single **convex bowl** — one minimum, so GD always reaches it.

Convexity is exactly why we can fit logistic regression by gradient descent with confidence (next slides).



Log-loss (cross-entropy)

Penalize by how much probability you gave the *true* label:

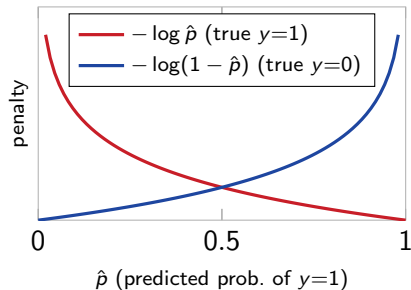
- ▶ true label $y = 1$: penalty = $-\log \hat{p}$
- ▶ true label $y = 0$: penalty = $-\log(1 - \hat{p})$

Both blow up to ∞ as you become confidently wrong, and go to 0 when you are confidently right.

One formula, where y picks the branch:

$$L(y, \hat{p}) = -[y \log \hat{p} + (1 - y) \log(1 - \hat{p})].$$

This is **cross-entropy** / Bernoulli loss – the same loss reappears in neural nets and boosting.



Where log-loss comes from: one formula, step by step

The **likelihood** is the probability the model assigns to the labels we actually saw.

1. One example, two cases: $P(y=1) = \hat{p}$, $P(y=0) = 1 - \hat{p}$.
2. Fold into **one** expression – the exponent switches the right term on:

$$P(y) = \hat{p}^y (1 - \hat{p})^{1-y} \quad (y=1 : \hat{p}^1(1-\hat{p})^0 = \hat{p}; \quad y=0 : \hat{p}^0(1-\hat{p})^1 = 1-\hat{p}).$$

3. All n rows are **independent** $\Rightarrow$ multiply:

$$\mathcal{L}(\theta) = \prod_{i=1}^n \hat{p}_i^{y_i} (1 - \hat{p}_i)^{1-y_i}.$$

4. **Log** (turns the product into a sum), then **negate** (so we minimize):

$$-\log \mathcal{L} = - \sum_{i=1}^n [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)] = \text{log-loss}.$$

Two names, one quantity: this is **maximum likelihood** (statistics) and the **cross-entropy** between the labels and the model's predictions (information theory).

Fitting logistic regression

We minimize the empirical risk (average log-loss):

$$\hat{\theta} = \arg \min_{\theta} \frac{1}{n} \sum_{i=1}^n - \left[y^{(i)} \log \sigma(\theta^\top \mathbf{x}^{(i)}) + (1 - y^{(i)}) \log(1 - \sigma(\theta^\top \mathbf{x}^{(i)})) \right].$$

- **Convex** in θ – no bad local minima.
- But **no closed-form** solution (unlike OLS) – fit **numerically** with an iterative optimizer (next slide). Convex $\Rightarrow$ it reaches the global optimum.

Separable-data trap. If a line perfectly splits the classes, the weights grow without bound (push $\hat{p} \rightarrow 0/1$ for more reward). The fix is the same as in regression: **regularization** (Ridge/Lasso) – in sklearn this is the C parameter.

The gradient: same shape as linear regression

Gradient descent needs the gradient of the log-loss. The sigmoid's tidy derivative $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ makes the chain rule collapse:

$$\nabla_{\theta} \frac{1}{n} \sum_{i=1}^n L(y^{(i)}, \hat{p}^{(i)}) = \frac{1}{n} \sum_{i=1}^n \left(\underbrace{\sigma(\theta^{\top} \mathbf{x}^{(i)})}_{\hat{p}^{(i)}} - y^{(i)} \right) \mathbf{x}^{(i)}.$$

Read it as **(prediction – truth) × feature**, averaged over the data:

- ▶ points we already predict well ($\hat{p} \approx y$) contribute ≈ 0 ;
- ▶ confident mistakes pull the hardest.

Identical to linear regression! There the gradient is $\frac{1}{n} \sum (\theta^{\top} \mathbf{x}^{(i)} - y^{(i)}) \mathbf{x}^{(i)}$ – same formula, only the prediction changes from $\hat{y} = \theta^{\top} \mathbf{x}$ to $\hat{p} = \sigma(\theta^{\top} \mathbf{x})$. The same GD machinery (Ch. 1) and L_2 weight decay carry straight over.

Plain gradient descent steps downhill along this – the next slide reaches the optimum in fewer steps.

How is it actually optimized?

The log-loss is **convex and smooth** – we get a gradient *and* curvature to exploit.

- ▶ Plain **gradient descent** works (Chapter 1), but needs many steps.
- ▶ sklearn's default is **L-BFGS** – a *quasi-Newton* method: it uses curvature via an *approximate* inverse Hessian built from recent gradients (never the full $p \times p$ matrix), so it converges in far fewer iterations and stays memory-light. Convex + smooth is exactly where Newton-type methods shine.

solver	when to use
lbfgs (default)	medium, dense data; L2
liblinear	small data; L1 — via <i>coordinate descent</i>
saga	large / sparse; L1 / elastic-net
newton-cholesky	$n \gg \text{\#features}$ (exact Hessian)

Why not plain GD? On a convex, smooth loss, curvature (second-order) info reaches the optimum in a handful of iterations, not hundreds. *Coordinate descent* – the Lasso optimizer – is also available; it is what liblinear uses, not the default here.

In practice

The theory is done. Now: the `sklearn` call, reading the coefficients as odds ratios, and how we will know the model is any good.

Logistic regression in sklearn

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

clf = make_pipeline(
    StandardScaler(),                                # scale features first
    LogisticRegression(C=1.0, class_weight="balanced"),
)
clf.fit(X_train, y_train)
proba = clf.predict_proba(X_test)[: , 1]           # P(bad), bad=1
```

argument	what it controls
C	inverse regularization strength (small C = stronger penalty)
penalty	"l2" (default), "l1" (sparse)
class_weight="balanced"	up-weights the rare class – cheese is 3% bad

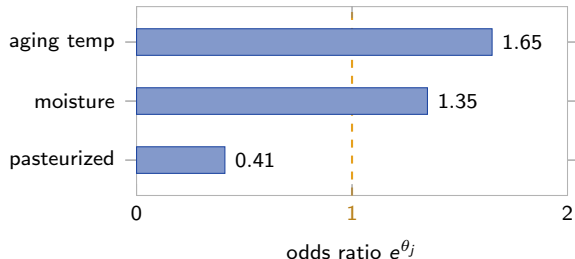
Scaling makes coef_ per standard deviation; for per-unit odds ratios (next slides), fit on raw features.

Reading the coefficients

Still **very interpretable** – though less *directly* than linear regression:

- ▶ θ_j = effect on the **log-odds** per +1 unit of feature j .
- ▶ e^{θ_j} = the **odds ratio**: multiply the odds by this.
- ▶ aging temp: $e^{0.5} = 1.65$ ($+1^\circ\text{C} \Rightarrow$ odds $\times 1.65$).
- ▶ moisture: 1.35 – wetter $\Rightarrow$ more likely bad.
- ▶ pasteurized: 0.41 – cuts the odds by $\sim 60\%$.

Mind the scale: in linear regression θ_j is the change in y itself; here it changes the **log-odds**, and the effect on the probability is non-linear (largest near $p = 0.5$). Read it as an odds ratio, not a direct effect. (Raw units; with `StandardScaler`, `coef_` is per standard deviation.)



Orange line = 1 (no effect): bars right raise the odds, left lower them.

Strengths, limits, and what's next

Strengths

- ▶ Interpretable (odds ratios).
- ▶ Outputs a probability/score.
- ▶ Fast; convex; great *baseline*.

Limits

- ▶ **Linear** decision boundary only.
- ▶ Assumes log-odds linear in features.
- ▶ Needs feature engineering (e.g. polynomial terms) for curved boundaries.

Still to address: how to *evaluate* a classifier honestly – accuracy alone is misleading on imbalanced data (next slide, then the **classification-metrics** lecture).

Wait – how do we know it's any good?

We can *build* a classifier now. But how do we **score** it?

Predict-first: cheese is 3% bad. A lazy model that calls every batch “good” scores. . . **97% accuracy** – while catching **zero** bad batches.

Accuracy is **misleading** on imbalanced data, and it ignores the asymmetric cost (a missed bad batch $\gg$ a needless re-check). Classification needs metrics that separate “*catches the bad ones*” from “*doesn't cry wolf*”: precision, recall, F1, ROC-AUC, PR-AUC – and the threshold c itself.

That is the whole next lecture – **classification metrics**, on the same cheese factory.

Beyond two classes

We lumped every defect as “bad”. Really a wheel is *good*, *moldy*, or *too dry* – now $K = 3$ classes, not 2. One sigmoid gives one probability; we need K of them that sum to 1.

Two ways to reach K classes

The target is now one of K **mutually exclusive** classes (a wheel is exactly one type):
cheese {good, moldy, dry}, digits 0–9, iris species.

1. **Reduce to binary** – reuse what we have:

- ▶ **One-vs-Rest (OvR)**: K “class k vs. the rest” classifiers.
- ▶ **One-vs-One (OvO)**: one classifier per *pair*, then vote.

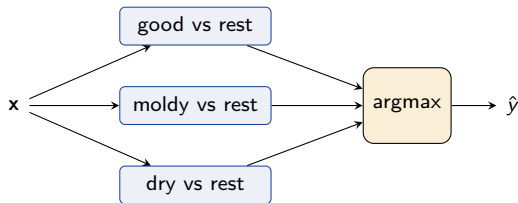
2. **Generalize the model** – one joint fit:

- ▶ **Softmax** (multinomial logistic regression):
 K scores, normalized to probabilities that sum to 1.

Both keep the model **linear**. We look at each, then pick a default.

One-vs-Rest (and One-vs-One)

Train K *binary* logistic classifiers, each “class k vs. the rest,” then predict the highest score: $\hat{y} = \arg \max_k \sigma(\boldsymbol{\theta}_k^\top \mathbf{x})$.



► ✓ dead simple; reuses binary; trivially parallel.

► ✗ K fits; scores not jointly calibrated; each sees imbalance.

One-vs-One: instead train $\binom{K}{2}$ pairwise classifiers and vote – more models, each on less data (the default for SVMs).

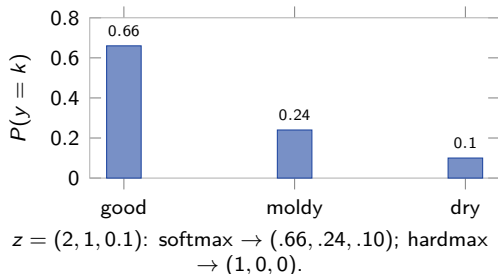
Softmax: a soft argmax

Give each class its own weights θ_k , score $z_k = \theta_k^\top \mathbf{x}$, then **normalize** the scores into probabilities:

$$P(y = k \mid \mathbf{x}) = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}$$

(the **softmax**).

- ▶ Each $P \geq 0$ and they **sum to 1** – a distribution over classes; predict $\hat{y} = \arg \max_k z_k$.
- ▶ **Soft version of hardmax.** A true **argmax** returns a one-hot spike — e.g. $(1, 0, 0)$ — a flat-then-jump function that is *not differentiable*.
- ▶ Softmax **smooths** it into probabilities leaning toward the largest score — differentiable, so we can train by gradient descent. Scale the scores up and softmax $\rightarrow$ the hard one-hot.



Softmax generalizes the sigmoid

Write the sigmoid's 1 as e^0 :

$$\sigma(z) = \frac{1}{1 + e^{-z}} = \frac{e^z}{e^z + 1} = \frac{e^z}{e^z + e^0}.$$

That is exactly **softmax over two scores** $\{z_1 = z, z_0 = 0\}$: class 1 carries the score z , class 0 is pinned at 0. The sigmoid is the $K = 2$ softmax with one score fixed.

Binary logistic regression is the two-class softmax. The log-loss, the $(\hat{p} - y) \mathbf{x}$ gradient, the linear boundary — all are the $K = 2$ special case of what we just built. One model, K classes.

Softmax: loss, gradient, and boundaries

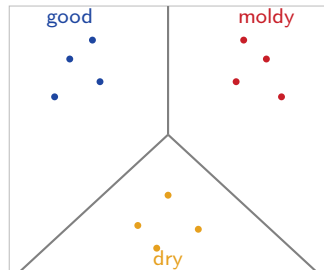
Loss – categorical cross-entropy (same MLE story, K branches):

$$-\sum_{i=1}^n \log P(y = y_i \mid \mathbf{x}^{(i)}).$$

Gradient – it keeps the same shape:

$$\nabla_{\theta_k} = \sum_{i=1}^n (P(k \mid \mathbf{x}^{(i)}) - 1[y_i = k]) \mathbf{x}^{(i)}.$$

Still **(prediction – truth) × feature**, where the truth $1[y_i=k]$ is the one-hot label (1 for the true class, else 0).
The binary gradient, one per class.



Boundaries ($z_i = z_j$) are **lines**: K convex regions.

Which one? And in sklearn

Softmax – the default choice:

- ▶ classes mutually exclusive;
- ▶ one joint fit, coherent probabilities (sum to 1).

One-vs-Rest – reach for when:

- ▶ you want a simple, reusable binary model;
- ▶ labels can overlap / need independent scores.

```
from sklearn.linear_model import LogisticRegression
clf = LogisticRegression(max_iter=1000)    # softmax automatic for  $K \geq 3$ 
clf.fit(X_train, y_train)                  #  $y$  in  $\{0, 1, \dots, K-1\}$ 
proba = clf.predict_proba(X_test)          # shape  $(n, K)$ ; each row
                                           # sums to 1

from sklearn.multiclass import OneVsRestClassifier    # explicit OvR,
                                                    if needed
ovr = OneVsRestClassifier(LogisticRegression()).fit(X_train, y_train)
```

Since sklearn 1.7 the `multi_class` argument is gone: `LogisticRegression` does softmax for $K \geq 3$ (solvers `lbfgs/saga/...`); `liblinear` is binary-only, so wrap it in `OneVsRestClassifier`.

Recap

- ▶ **Classification** predicts a category; a good classifier outputs a **score** in $[0, 1]$, then you **threshold**.
- ▶ **Odds** $= p/(1 - p)$; **log-odds** maps $(0, 1) \rightarrow \mathbb{R}$.
- ▶ **Logistic regression** = a linear model on the log-odds, squashed to a probability by the **sigmoid** $\sigma(\theta^\top \mathbf{x})$. Linear boundary.
- ▶ Fit by minimizing **log-loss** (cross-entropy) with gradient descent; regularize via C.
- ▶ Coefficients read as **odds ratios** (e^{θ_j}).
- ▶ **Multiclass**: **softmax** (multinomial) or one-vs-rest – softmax generalizes the sigmoid; the K probabilities sum to 1.

Workflow: scale features $\rightarrow$ fit logistic regression as your baseline $\rightarrow$ read the odds ratios $\rightarrow$ evaluate with the right metric (next lecture) $\rightarrow$ tune the threshold for your costs.